

Ecommerce Project - Full Terraform & Deployment Documentation

1. Project Overview

This project demonstrates a full CI/CD pipeline for an EKS-based Ecommerce application using Terraform, AWS, and Docker containers. It includes:

- EKS cluster setup
- Node group creation
- IAM roles and policies
- Terraform infrastructure as code
- Deployment using `kubect1`
- Diagram of CI/CD and deployment flow

Note: All sensitive information (AWS Account IDs, access keys) has been removed or masked.

2. Terraform Files

2.1 provider.tf

```
provider "aws" {  
  region = "ap-south-1"  
}
```

2.2 backend.tf

```
terraform {  
  backend "s3" {  
    bucket = "your-terraform-state-bucket"  
    key    = "terraform.tfstate"  
    region = "ap-south-1"  
  }  
}
```

2.3 vpc.tf

```
resource "aws_vpc" "this" {  
  cidr_block = "10.0.0.0/16"
```

```

    tags = { Name = "eks-vpc" }
}

resource "aws_subnet" "public" {
  count          = 2
  vpc_id         = aws_vpc.this.id
  cidr_block     = "10.0.${count.index}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = { Name = "eks-public-subnet-${count.index}" }
}

```

2.4 iam_roles.tf

```

# IAM Role for EKS Cluster
resource "aws_iam_role" "eks_cluster_role" {
  name = "eks-cluster-role"
  assume_role_policy = data.aws_iam_policy_document.eks_cluster_assume_role.json
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  role      = aws_iam_role.eks_cluster_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

resource "aws_iam_role_policy_attachment" "eks_vpc_policy" {
  role      = aws_iam_role.eks_cluster_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSVPCResourceController"
}

# IAM Role for EKS Node Group
resource "aws_iam_role" "eks_node_role" {
  name = "eks-node-role"
  assume_role_policy = data.aws_iam_policy_document.eks_worker_assume_role.json
}

resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "eks_ecr_policy" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {

```

```

    role      = aws_iam_role.eks_node_role.name
    policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

```

2.5 eks_cluster.tf

```

resource "aws_eks_cluster" "this" {
  name      = "my-eks-cluster"
  version   = "1.34"
  role_arn  = aws_iam_role.eks_cluster_role.arn

  vpc_config {
    subnet_ids = [for subnet in aws_subnet.public : subnet.id]
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy,
    aws_iam_role_policy_attachment.eks_vpc_policy
  ]
}

```

2.6 nodegroup.tf

```

resource "aws_security_group" "eks_node_sg" {
  name     = "eks-node-sg"
  vpc_id   = aws_vpc.this.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "eks-node-sg" }
}

resource "aws_eks_node_group" "this" {

```

```

cluster_name      = aws_eks_cluster.this.name
node_group_name   = "my-node-group"
node_role_arn     = aws_iam_role.eks_node_role.arn
subnet_ids       = [for subnet in aws_subnet.public : subnet.id]

scaling_config {
  desired_size = 2
  max_size     = 3
  min_size     = 1
}

instance_types = ["t4g.small"]

remote_access {
  ec2_ssh_key           = "VijayKey"
  source_security_group_ids = [aws_security_group.eks_node_sg.id]
}

ami_type = "AL2023_ARM_64_STANDARD"

labels = { environment = "dev" }

depends_on = [
  aws_iam_role_policy_attachment.eks_worker_node_policy,
  aws_iam_role_policy_attachment.eks_ecr_policy,
  aws_iam_role_policy_attachment.eks_cni_policy
]
}

```

2.7 outputs.tf

```

output "eks_cluster_arn" {
  value = aws_eks_cluster.this.arn
}

output "eks_cluster_name" {
  value = aws_eks_cluster.this.name
}

output "eks_cluster_endpoint" {
  value = aws_eks_cluster.this.endpoint
}

output "node_group_arn" {
  value = aws_eks_node_group.this.arn
}

```

2.8 variables.tf

```
variable "region" {  
    default = "ap-south-1"  
}
```

3. Terraform Commands

```
terraform init  
terraform plan  
terraform apply -auto-approve
```

4. Kubernetes Deployment Example

```
aws eks update-kubeconfig --region ap-south-1 --name my-eks-cluster  
kubectl create deployment hello-world --image=nginx  
kubectl get pods  
kubectl expose deployment hello-world --type=LoadBalancer --port=80  
kubectl get svc
```

5. Project Diagram

CI/CD Deployment Pipeline

6. Notes

- AWS account ID and sensitive keys are masked.
- Node group is required to run workloads; without it, EKS cluster cannot host pods.
- Terraform code is fully inline; can be copied directly into `/workspace/infra`.
- Diagram is embedded in all document outputs.